BÀI THỰC HÀNH

LAB 08

Đo nhiệt độ với DS18B20

Giám sát nhiệt độ số qua giao tiếp 1-Wire bằng EduFramework

S32K144 · MaaZEDU

EduFramework

FPT University

Mục lục

1 Giới thiệu . . . 2
1.1 Tổng quan bài lab . . . 2
1.2 Mục tiêu . . . 2
2 Kiến thức nền . . . 2
2.1 Cảm biến nhiệt độ số DS18B20 . . . 2
2.2 Giao tiếp 1-Wire và đường dữ liệu DQ . . . 3
2.3 Từ lệnh đo đến giá trị nhiệt độ . . . 3
2.4 Độ phân giải và thời gian chuyển đổi . . . 4
3 Thiết lập phần cứng . . . 4
3.1 Phần cứng sử dụng . . . 4
3.2 Ánh xạ chân và kết nối module . . . 4
4 EduFramework API . . . 5
4.1 `DS18B20_begin()` . . . 5
4.2 `DS18B20_readCelsius()` . . . 6
5 Bài thực hành . . . 7
5.1 Yêu cầu . . . 7
5.2 Chương trình . . . 7
5.3 Kiểm chứng . . . 9
6 Mở rộng . . . 9
6.1 Thay đổi độ phân giải . . . 9
6.2 Đọc nhiệt độ theo cách không chặn . . . 10
6.3 Nhiều DS18B20 trên cùng bus 1-Wire . . . 10
6.4 Các tính năng nâng cao khác . . . 10
7 Tài liệu tham khảo . . . 11

1 Giới thiệu

1.1 Tổng quan bài lab

Đo nhiệt độ là chức năng phổ biến trong hệ thống nhúng, từ giám sát môi trường đến điều khiển thiết bị theo điều kiện nhiệt. Một ứng dụng có thể sử dụng một cảm biến tại một vị trí hoặc nhiều cảm biến phân bố trên cùng hệ thống. Trong nghiên cứu về mạng đo phân bố nhiệt độ không khí, Elyounsi và Kalashnikov đã đánh giá DS18B20 cùng cảm biến Pt100 và chỉ ra tầm quan trọng của điều kiện không khí, thời gian đáp ứng và cách bố trí phép đo khi so sánh kết quả [2] .

DS18B20 là cảm biến nhiệt độ số sử dụng giao thức `1-Wire` . Khác với phần tử cảm biến analog phải đưa điện áp tới bộ ADC của vi điều khiển, DS18B20 thực hiện phép chuyển đổi nhiệt độ bên trong linh kiện rồi cung cấp dữ liệu số qua đường truyền `DQ` . Cảm biến hỗ trợ độ phân giải từ `9 bit` đến `12 bit` và có mã nhận dạng riêng `64 bit` để phục vụ các ứng dụng nhiều thiết bị [1] .

Bài lab này sử dụng **module DS18B20 ba chân** cùng MaaZEDU Development Board. EduFramework cung cấp Device API để ứng dụng không phải tự tạo các xung thời gian của giao thức `1-Wire` . Chương trình khởi tạo một cảm biến, đọc nhiệt độ ở đơn vị độ C và hiển thị định kỳ trên Serial Monitor. Những thao tác quản lý nhiều thiết bị, thay đổi độ phân giải và đọc không chặn được dành cho phần Mở rộng [5] .

1.2 Mục tiêu

MỤC TIÊU

Các mục tiêu của bài lab bao gồm:

1. Mô tả đặc điểm của DS18B20 và phân biệt kết quả đo nhiệt độ số với tín hiệu analog được đọc qua ADC.
2. Giải thích vai trò của đường dữ liệu `DQ` , điện trở kéo lên và mã ROM trong giao tiếp `1-Wire` .
3. Phân biệt độ phân giải với độ chính xác và nhận biết sự phụ thuộc của thời gian chuyển đổi vào độ phân giải cảm biến.
4. Sử dụng `DS18B20_begin()` và `DS18B20_readCelsius()` để thu nhận dữ liệu nhiệt độ bằng EduFramework.
5. Xây dựng, kiểm chứng ứng dụng giám sát nhiệt độ và nhận biết trường hợp khởi tạo hoặc đọc cảm biến không thành công.

2 Kiến thức nền

2.1 Cảm biến nhiệt độ số DS18B20

DS18B20 tích hợp chức năng cảm biến và chuyển đổi nhiệt độ thành dữ liệu số. Vi điều khiển yêu cầu một phép đo, sau đó nhận dữ liệu nhiệt độ đã được mã hóa qua giao diện số. Theo datasheet, cảm biến có dải đo từ `-55 °C` đến `+125 °C` , độ chính xác công bố `±0.5 °C` trong khoảng `-10 °C` đến `+85 °C` và điện áp cấp nguồn ngoài từ `3.0 V` đến `5.5 V` [1] .

Đặc tính	Thông số theo datasheet DS18B20
Đại lượng đo	Nhiệt độ theo độ C.
Dải đo	`-55 °C` đến `+125 °C`.
Độ chính xác	`±0.5 °C` trong khoảng `-10 °C` đến `+85 °C`.
Độ phân giải	`9 bit`, `10 bit`, `11 bit` hoặc `12 bit`.
Giao tiếp	`1-Wire`, sử dụng một đường dữ liệu `DQ` và GND chung.
Nhận dạng	Mỗi cảm biến có một mã ROM riêng `64 bit`.

Bảng 8.2.1. Các đặc tính chính của DS18B20

Trong Lab 04 và Lab 06, ứng dụng làm việc với giá trị ADC của một tín hiệu analog. Đối với DS18B20, kết quả không được lấy bằng `analogRead()` mà thông qua một chuỗi lệnh và dữ liệu số của cảm biến. Vì vậy, chương trình có thể làm việc trực tiếp với nhiệt độ thay vì thực hiện phép quy đổi từ điện áp đầu vào [1] [5].

Ghi chú

Độ phân giải *mô tả bước thay đổi nhỏ nhất của dữ liệu nhiệt độ được biểu diễn.* ***Độ chính xác*** *liên quan tới mức sai lệch so với nhiệt độ thực hoặc giá trị tham chiếu. Việc hiển thị nhiều chữ số thập phân không bảo đảm phép đo có độ chính xác tương ứng [1].*

2.2 Giao tiếp 1-Wire và đường dữ liệu DQ

`1-Wire` là giao thức truyền thông nối tiếp trong đó master và các thiết bị trên bus dùng chung một đường dữ liệu. Các thiết bị cùng sử dụng đường GND làm mốc điện áp. Đường dữ liệu hoạt động theo nguyên tắc kéo xuống mức thấp rồi nhả đường truyền; điện trở pull-up đưa bus trở về mức cao khi không có thiết bị nào kéo xuống. Do đó, điện trở kéo lên là một phần cần thiết của kết nối `1-Wire` [3].

Một phiên trao đổi thông thường bắt đầu bằng thao tác reset từ master và phản hồi presence từ thiết bị, sau đó là các lệnh lựa chọn thiết bị và lệnh chức năng. Việc truyền một bit được thực hiện trong các khoảng thời gian (time slot) được quy định. Tài liệu kỹ thuật của Analog Devices trình bày bốn thao tác nền tảng: reset, ghi bit `0`, ghi bit `1` và đọc bit [3].

Đối với DS18B20, mỗi cảm biến còn có một mã ROM `64 bit` riêng. Mã này giúp master phân biệt các cảm biến khi nhiều thiết bị được nối song song trên cùng bus. Vì vậy, một đường dữ liệu có thể phục vụ nhiều điểm đo, với điều kiện mạng dây và giao tiếp đáp ứng yêu cầu điện của `1-Wire` [1].

Bài thực hành chính chỉ sử dụng một module nên không cần tự tìm và lưu mã ROM. `DS18B20_begin()` thực hiện việc tìm cảm biến và thiết lập kết nối thông qua OneWire driver của EduFramework [5].

2.3 Từ lệnh đo đến giá trị nhiệt độ

Để thu nhận một kết quả mới, DS18B20 phải thực hiện quá trình chuyển đổi nhiệt độ. Theo datasheet, lệnh `Convert T` bắt đầu chuyển đổi; khi hoàn thành, dữ liệu nhiệt độ được lưu trong **scratchpad**, vùng nhớ cũng chứa các byte cấu hình và byte kiểm tra `CRC8`. Dữ liệu nhiệt độ có thể được đọc bằng lệnh `Read Scratchpad` [1].

Trong EduFramework, các thao tác này được đóng gói trong hàm `DS18B20_readCelsius()`: bắt đầu chuyển đổi, chờ kết quả, đọc scratchpad, xác minh `CRC8` và giải mã dữ liệu thành nhiệt độ theo độ C. Ứng dụng nhận một giá trị `float` mà không phải thao tác với từng bit của giao thức [5].

Luồng thu nhận dữ liệu của bài lab được tóm tắt như sau:

`Nhiệt độ` → `DS18B20` → `1-Wire (DQ)` → `DS18B20_readCelsius()` → `Serial1` → `Serial Monitor`.

2.4 Độ phân giải và thời gian chuyển đổi

Độ phân giải của DS18B20 có thể được chọn từ `9 bit` đến `12 bit`. Theo thông số của nhà sản xuất, khi độ phân giải tăng, bước lượng tử hóa giảm nhưng thời gian chuyển đổi tối đa tăng. Bảng sau thể hiện các mức được quy định trong datasheet [1].

Độ phân giải	Bước nhiệt độ	Thời gian chuyển đổi tối đa
`9 bit`	`0.5 °C`	`93.75 ms`
`10 bit`	`0.25 °C`	`187.5 ms`
`11 bit`	`0.125 °C`	`375 ms`
`12 bit`	`0.0625 °C`	`750 ms`

Bảng 8.2.2. Độ phân giải và thời gian chuyển đổi của DS18B20

Một giá trị như `28.063 °C` trên Serial Monitor có thể tương ứng với nhiệt độ được mã hóa ở bước `0.0625 °C` rồi được làm tròn khi in. Không nên dùng số chữ số thập phân để suy ra độ chính xác của cảm biến. Ngoài ra, thời gian đọc ứng dụng còn có phần giao tiếp và xử lý của driver, nên chu kỳ hiển thị không bằng chính xác thời gian chuyển đổi trong bảng.

3 Thiết lập phần cứng

3.1 Phần cứng sử dụng

Bài thực hành sử dụng MaaZEDU Development Board với vi điều khiển `S32K144` và một module DS18B20 ba chân. Trên module đang sử dụng, các chân kết nối được ký hiệu `+`, `OUT` và `-`; tương ứng với nguồn dương, đường dữ liệu và GND. Chương trình đã được kiểm chứng với nguồn `3.3 V` và một cảm biến kết nối tới `GPIO7` [6] [5].

Phần cứng	Mô tả
MaaZEDU Development Board	Board sử dụng vi điều khiển S32K144 và cung cấp các chân GPIO.
DS18B20 Temperature Sensor Module	Module ba chân có cảm biến DS18B20 và các linh kiện phụ trợ trên PCB.
Jumper wires	Ba dây nối nguồn, GND và dữ liệu giữa module với MaaZEDU.
USB Cable	Kết nối board với máy tính để nạp chương trình và sử dụng Serial Monitor.

Bảng 8.3.1. Phần cứng sử dụng trong bài thực hành

3.2 Ánh xạ chân và kết nối module

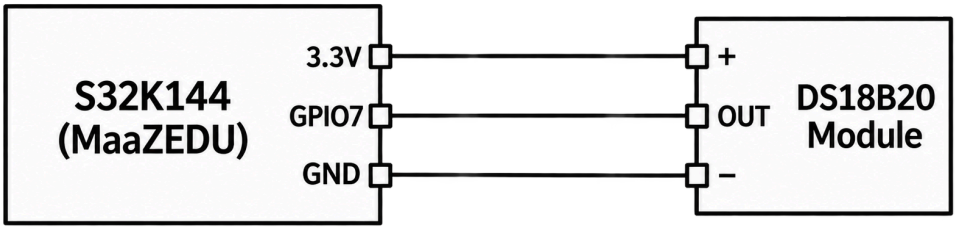
Tài liệu MaaZEDU Development Board Guide sử dụng ký hiệu `GPIO7` cho một chân GPIO đưa ra đầu nối. Trong ứng dụng EduFramework, `GPIO7` là **Logical Pin** được truyền vào API để chọn đường dữ liệu `1-Wire`.

Thành phần	Logical Pin	MCU Pin	Chức năng
DS18B20 `OUT`	`GPIO7`	`PTD9`	Digital I/O / 1-Wire data; tên cổng MCU theo MaaZEDU Guide 1.0.

Bảng 8.3.2. Chân dữ liệu sử dụng trong Lab 08

Module DS18B20	MaaZEDU	Chức năng
`+`	`3.3V`	Cấp nguồn ngoài cho cảm biến.
`OUT`	`GPIO7`	Truyền và nhận dữ liệu `1-Wire`.
`-`	`GND`	Tạo mốc điện áp chung.

Bảng 8.3.3. Kết nối ba chân của module DS18B20



Hình 8.3.1. Sơ đồ kết nối module DS18B20 với MaaZEDU Development Board

Module của bài thực hành sử dụng các linh kiện phụ trợ đã gắn trên PCB. Với mạch `1-Wire`, cần có đường pull-up thích hợp giữa đường dữ liệu và nguồn. Nếu thay bằng một DS18B20 rời hoặc module khác, phải kiểm tra sơ đồ thực tế và bổ sung điện trở pull-up ngoài khi cần; không được mặc định mọi module đều có cùng cấu hình điện trở [3].

> **CẢNH BÁO**
>
> Ngắt nguồn trước khi thay đổi dây nối. Đối chiếu đúng ký hiệu `+`, `OUT`, `-` trên module đang sử dụng; không suy luận chân từ màu dây hoặc từ thứ tự chân của IC DS18B20 dạng TO-92. Bài lab sử dụng nguồn `3.3 V` để tương thích với mức logic của board.

4 EduFramework API

EduFramework v3.1.1 cung cấp hai nhóm hàm cho DS18B20: **Simple API** dùng cho một cảm biến và **Advanced API** để quản lý bus cùng thiết bị cụ thể. Bài thực hành chính chỉ sử dụng hai hàm `DS18B20_begin()` và `DS18B20_readCelsius()`; các thao tác chi tiết về ROM, scratchpad và độ phân giải được dành cho phần Mở rộng [5].

4.1 `DS18B20_begin()`

`DS18B20_begin()` cấu hình đường GPIO và tìm một cảm biến DS18B20 trên bus `1-Wire`. Khi tìm được cảm biến, driver kiểm tra dữ liệu và chuẩn bị thiết bị cho các phép đọc tiếp theo. API nhận một Logical Pin có khả năng Digital I/O [5].

DS18B20_begin

Khởi tạo kết nối `1-Wire` và tự động chọn cảm biến DS18B20 đầu tiên được tìm thấy trên chân được chỉ định.

Cú pháp

```
DS18B20_begin(pin);
```

Tham số

Tham số	Kiểu / giá trị chấp nhận	Mô tả
pin	Digital Logical Pin	Chân nối với `OUT` của module DS18B20, ví dụ `GPIO7`.

Giá trị trả về

`true` khi tìm thấy và khởi tạo cảm biến thành công; `false` nếu thao tác khởi tạo hoặc kiểm tra thiết bị thất bại.

Ví dụ:

```
bool ready = DS18B20_begin(GPIO7);
```

Trong bài thực hành, kết quả trả về được kiểm tra ngay sau khi khởi tạo. Chương trình không tiếp tục đọc nhiệt độ nếu cảm biến chưa sẵn sàng.

4.2 `DS18B20_readCelsius()`

`DS18B20_readCelsius()` bắt đầu phép chuyển đổi nhiệt độ, chờ cảm biến hoàn thành, sau đó đọc và giải mã kết quả. Đây là phép đọc **blocking**: lời gọi chưa kết thúc khi cảm biến vẫn đang thực hiện chuyển đổi [5].

DS18B20_readCelsius

Thực hiện một phép đo nhiệt độ mới và trả về kết quả theo độ C.

Cú pháp

```
DS18B20_readCelsius();
```

Giá trị trả về

Nhiệt độ theo độ C khi thành công; `NAN` khi xảy ra lỗi. `NAN` không phải một giá trị nhiệt độ hợp lệ.

Ví dụ:

```
float temperature = DS18B20_readCelsius();
```

Để phân biệt dữ liệu hợp lệ với kết quả lỗi, chương trình sử dụng `isnan()` từ `math.h`. Hàm này kiểm tra giá trị **Not a Number** (`NAN`) trước khi dữ liệu được hiển thị lên Serial Monitor. Việc tra cứu mã lỗi chi tiết bằng `DS18B20_lastStatus()` sẽ được giới thiệu ở phần Mở rộng.

Các thao tác `Serial1_begin()`, `Serial1_print()`, `Serial1_println()` và `Serial1_printFloat()` đã thuộc nhóm Serial API của EduFramework và được sử dụng lại từ Lab 03. Lab 08 không lặp lại tài liệu cú pháp chi tiết của nhóm API này [5].

5 Bài thực hành

5.1 Yêu cầu

Xây dựng chương trình giám sát nhiệt độ bằng DS18B20 và hiển thị kết quả qua Serial Monitor. Chương trình được thực hiện với các điều kiện sau:

- Sử dụng module DS18B20 trên `GPIO7` và nguồn `3.3 V`.
- Khởi tạo `Serial1` với baud rate `9600` và hiển thị thông báo bắt đầu.
- Kiểm tra việc khởi tạo cảm biến trước khi đọc dữ liệu.
- Đọc nhiệt độ theo độ C; chỉ hiển thị nhiệt độ khi dữ liệu hợp lệ.
- Sau mỗi lần đọc, chờ `1000 ms` rồi lặp lại.

Khi hệ thống hoạt động ổn định, thay đổi nhẹ nhiệt độ tại đầu cảm biến bằng cách chạm tay hoặc đưa gần một nguồn nhiệt an toàn. Quan sát cả quá trình nhiệt độ tăng và giảm trở lại khi bỏ nguồn nhiệt. Không yêu cầu cảm biến phải hiển thị một giá trị cố định, vì điều kiện môi trường và vị trí cảm biến có thể khác nhau [2].

5.2 Chương trình

Trong `src/main.c`, triển khai chương trình đã được kiểm chứng trên phần cứng như sau:

```c
#include "Arduino.h"
#include "ds18b20.h"

#include <math.h>

int main(void)
{
    float temperature = 0.0F;

    setup();

    Serial1_begin(9600U);
    Serial1_println("DS18B20 Temperature Monitor");

    if (false == DS18B20_begin(GPIO7))
    {
        Serial1_println("DS18B20 initialization failed!");

        while (1)
        {
            delay(1000U);
        }
    }

    Serial1_println("DS18B20 initialized successfully.");

    while (1)
    {
        temperature = DS18B20_readCelsius();

        if (false == isnan(temperature))
        {
            Serial1_print("Temperature: ");
            Serial1_printFloat(temperature);
            Serial1_println(" C");
        }
        else
        {
            Serial1_println("Temperature reading failed!");
        }

        delay(1000U);
    }

    return 0;
}
```

Mã nguồn 8.5.1. Chương trình giám sát nhiệt độ bằng module DS18B20

`setup()` khởi tạo các thành phần nền tảng của EduFramework; các cơ chế clock, timer và UART trên họ S32K1xx được NXP mô tả trong AN5413 [4] . `Serial1_begin(9600U)` thiết lập kênh Serial Monitor, còn `DS18B20_begin(GPIO7)` khởi tạo bus và chọn cảm biến. Nếu khởi tạo thất bại, chương trình hiển thị thông báo lỗi và không thực hiện vòng lặp đo.

Trong vòng lặp chính, `DS18B20_readCelsius()` thực hiện phép đo mới. Kết quả được kiểm tra bằng `isnan()`; giá trị hợp lệ được hiển thị bằng `Serial1_printFloat()`, còn trường hợp lỗi được thông báo riêng. Sau đó, `delay(1000U)` tạo khoảng nghỉ trước lần đọc tiếp theo.

Ghi chú

*`delay(1000U)` là thời gian chờ **sau** mỗi lần đọc, không phải chu kỳ đo chính xác `1000 ms`. Ở độ phân giải `12 bit`, một phép chuyển đổi DS18B20 có thể mất tới `750 ms` theo datasheet; tổng khoảng cách giữa các lần hiển thị còn có thêm thời gian giao tiếp và xử lý [1] [5].*

5.3 Kiểm chứng

Build và nạp chương trình xuống MaaZEDU Development Board, sau đó mở Serial Monitor với baud rate `9600`. Đối chiếu các điều kiện kiểm chứng:

- Màn hình hiển thị thông báo khởi động và thông báo khởi tạo thành công.
- Nhiệt độ được cập nhật liên tục, không có giá trị `NAN` được in như dữ liệu đo.
- Khi điều kiện nhiệt ổn định, các giá trị liên tiếp nằm trong một vùng gần nhau.
- Khi làm ấm cảm biến, kết quả có xu hướng tăng; khi để nguội, kết quả giảm dần.

Một đoạn dữ liệu đã thu được trong thử nghiệm phần cứng của bài lab:

```
DS18B20 Temperature Monitor
DS18B20 initialized successfully.
Temperature: 28.063 C
Temperature: 28.063 C
Temperature: 28.000 C
Temperature: 28.000 C
Temperature: 27.938 C
Temperature: 27.813 C
```

Các giá trị trên là kết quả của một lần thử nghiệm, không phải mức nhiệt chuẩn bắt buộc đạt được. Theo nghiên cứu của Elyounsi và Kalashnikov, điều kiện trao đổi nhiệt và thời gian đáp ứng có thể ảnh hưởng tới việc so sánh số đo của các cảm biến đặt gần nhau; vì vậy không được xem phép thử này là quy trình hiệu chuẩn [2].

KẾT QUẢ MONG ĐỢI

Module DS18B20 được khởi tạo thành công và Serial Monitor hiển thị nhiệt độ theo độ C qua các lần đọc liên tiếp. Khi nhiệt độ tại cảm biến thay đổi, số liệu hiển thị phản ánh được xu hướng thay đổi tương ứng. Chương trình không sử dụng giá trị đọc lỗi làm kết quả nhiệt độ.

6 Mở rộng

Simple API phù hợp với ứng dụng dùng một cảm biến và một luồng đo tuần tự. DS18B20 còn hỗ trợ các chức năng khác ở cấp giao thức và trong Advanced API của EduFramework. Các nội dung sau là hướng khảo sát mở rộng, không phải yêu cầu bắt buộc của chương trình chính [1] [5].

6.1 Thay đổi độ phân giải

Theo datasheet, DS18B20 cho phép cấu hình độ phân giải từ `9 bit` đến `12 bit`. Giảm độ phân giải làm giảm thời gian chuyển đổi tối đa nhưng tăng bước biểu diễn nhiệt độ. Trong EduFramework, `DS18B20_SetResolution()` thuộc Advanced API, nhận một thiết bị đã được chọn và mức độ phân giải cần thiết lập.

Các mức được định nghĩa sẵn gồm `DS18B20_RESOLUTION_9_BIT`, `DS18B20_RESOLUTION_10_BIT`, `DS18B20_RESOLUTION_11_BIT` và `DS18B20_RESOLUTION_12_BIT` [5].

Mở rộng: So sánh thời gian cập nhật và bước thay đổi nhiệt độ ở hai mức `9 bit` và `12 bit`. Để gọi API cấu hình cần sử dụng Advanced API nhằm tạo bus context, tìm ROM và chọn thiết bị; không thể truyền trực tiếp Logical Pin vào `DS18B20_SetResolution()`.

6.2 Đọc nhiệt độ theo cách không chặn

Trong chương trình chính, `DS18B20_readCelsius()` chờ phép chuyển đổi hoàn thành. Một ứng dụng phải xử lý thêm nút nhấn, hiển thị hoặc các tác vụ theo thời gian có thể cần tách việc bắt đầu chuyển đổi khỏi việc đọc kết quả. Advanced API hỗ trợ chuỗi thao tác:
`DS18B20_StartConversion()` → `DS18B20_ConversionReady()` → `DS18B20_ReadLastCelsius()`.
Hàm đầu bắt đầu phép chuyển đổi, hàm thứ hai kiểm tra trạng thái sẵn sàng, và hàm cuối đọc giá trị đã chuyển đổi. Chỉ đọc kết quả sau khi phép chuyển đổi hoàn tất. Cách tiếp cận này có thể được kết hợp với `millis()` để tổ chức chương trình không chặn, nhưng việc sử dụng các hàm Advanced API đòi hỏi quản lý `DS18B20_Bus_t` và `DS18B20_Device_t` [5].

Mở rộng: Thiết kế ứng dụng tiếp tục thay đổi trạng thái một LED trong lúc cảm biến đang chuyển đổi nhiệt độ; không dùng `delay()` để chờ hoàn tất.

6.3 Nhiều DS18B20 trên cùng bus 1-Wire

Mỗi DS18B20 có mã ROM `64 bit` riêng, nhờ đó master có thể nhận dạng nhiều cảm biến cùng nối với một đường dữ liệu. Đây cũng là đặc điểm được quan tâm trong nghiên cứu các mạng đo phân bố nhiệt độ [1] [2].
Trong EduFramework, `DS18B20_FindDevices()` liệt kê mã ROM, còn `DS18B20_Attach()` gắn một thiết bị đã tìm được với device context. Khác với Simple API, cách sử dụng này cho phép chọn đúng cảm biến cần đọc khi có nhiều thiết bị trên cùng bus [5].

Mở rộng: Nếu có ít nhất hai DS18B20, khảo sát mã ROM của từng cảm biến và xây dựng chương trình hiển thị nhiệt độ theo từng ID. Cần kiểm tra nguồn cấp, đường pull-up và giới hạn vật lý của mạng dây trước khi mở rộng bus.

6.4 Các tính năng nâng cao khác

EduFramework v3.1.1 còn cung cấp những chức năng phục vụ kiểm tra và cấu hình thiết bị:

- `DS18B20_lastStatus()` và `DS18B20_StatusString()` hỗ trợ xác định và hiển thị nguyên nhân lỗi, chẳng hạn không tìm thấy thiết bị hoặc CRC không hợp lệ.
- `DS18B20_ReadScratchpad()` cho phép quan sát các byte dữ liệu của cảm biến; driver xác minh byte CRC8 trước khi chấp nhận kết quả.
- `DS18B20_SetAlarmThresholds()` và `DS18B20_FindAlarms()` hỗ trợ thiết lập ngưỡng và tìm thiết bị trong trạng thái cảnh báo.
- `DS18B20_CopyScratchpad()` và `DS18B20_RecallEEPROM()` phục vụ lưu và khôi phục cấu hình trong EEPROM của cảm biến.

Những tính năng này tương ứng với các khả năng ROM, scratchpad, CRC và alarm được mô tả trong datasheet [1] [5].

Ghi chú

Driver OneWire hiện tại của EduFramework yêu cầu cấp nguồn ngoài qua chân `VDD` khi đo nhiệt độ hoặc ghi EEPROM; chế độ parasite power cần strong pull-up chưa được hỗ trợ. Không thực hiện thao tác ghi EEPROM liên tục trong vòng lặp do bộ nhớ có giới hạn chu kỳ ghi [1] [5].

7 Tài liệu tham khảo

[1] Maxim Integrated / Analog Devices, *DS18B20 - Programmable Resolution 1-Wire Digital Thermometer* , DS18B20, Rev. 6, 2019.

[2] Ali Elyounsi and Alexander N. Kalashnikov, *Evaluating Suitability of a DS18B20 Temperature Sensor for Use in an Accurate Air Temperature Distribution Measurement Network* , Engineering Proceedings, 10(1), 56, 2021.

[3] Maxim Integrated / Analog Devices, *1-Wire Communication Through Software* , Analog Devices Technical Article, 2002.

[4] NXP Semiconductors, *S32K1xx Series Cookbook* , AN5413, Rev. 5, 2020.

[5] EduFramework, *DS18B20 Device API and OneWire Driver* , EduFramework Source Code, v3.1.1.

[6] FPT Software, *MaaZEDU Development Board Guide* , Rev. 1.0, 2025.